Categorical Imperative Programming

Type theory, refinement, and semantics for SSA

Jad-Elkhaleq Ghalayini

March 2026

UNIVERSITY OF CAMBRIDGE
Department of Computer Science and Technology

This thesis is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the preface and specified in the text. It is not substantially the same as any work that has already been submitted, or is being concurrently submitted, for any degree, diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the preface and specified in the text. It does not exceed the prescribed word limit for the relevant Degree Committee.

Contents

1. Introduction

The fundamental question of this thesis is: *what does a computer program mean?*

We can approach this in two ways: from the bottom-up or the top-down.

The former tends to be more natural for computer scientists.

A computer is a machine for following instructions – a program is a sequence of instructions for it to follow.

In particular, a computer has an *instruction set* – a finite set of *operations* which it can perform, which read from and write to its *state* – that is, the current state of the registers, program counters, memory, interrupts, and all the other bits which make a modern CPU tick.

Which is a *lot*.

This thesis is about the *denotational semantics* of the *static single assignment* – or *SSA* – compiler *intermediate representation* – or *IR*.

I want to start by justifying this choice of topic.

- *Why* do we need a semantics for a *compiler IR*?

  Because high-level languages have too many features to effectively study *or* to lower in one pass.

  Because low-level languages are designed to be *executed* efficiently – making them difficult both to reason about or to lower in one pass.

- *Why* should we use *SSA* as our compiler IR?

  Practically: most modern compilers use it.

  Theoretically: it's right at the intersection of functional and imperative programming – letting us use it as a *thin waist* to develop the *isomorphism* between the two.

- *Why* should we give a *denotational* semantics?

  Because we want to reason about programs *algebraically* – and to do so in a *compositional* manner.

We now want to give a formal account of the type-theoretic presentation of SSA we sketched in the introduction. In particular, for each of the intermediate representations we wish to study, we need to provide:

- A *grammar*, to enumerate the syntactically well-formed terms in our language – we've already provided this in the introduction for most of our intermediate representations.
- A *typing relation*, which allows us to determine whether a program is *semantically* well-formed.

In particular, we'll need a *type system* to construct our typing relation with respect to – later, we'll be able to relate different intermediate representations by giving them different typing relations over the same type system.

2. S-Expressions

TODO 1: This include should shift heading levels down by 1

3. Evaluating Arithmetic Expressions

TODO 2: Can pull this bit up into the intro, replacing the old text?

TODO 3: "a traditional pattern" ==> "the traditional pattern of operational semantics"

Our contributions:

- **Denotational instead (complete w.r.t. operational – need to prove!)**
- **Completeness of equational theory w.r.t. denotational semantics**
- **Treatment of effects**
- ***Refinement* theory – completeness of *refinement* w.r.t. denotational semantics**

TODO 4: Think about rewriting the below without any operational semantics at all – explain the shared bit of the pattern as "most/many/the first few of the chapters start out..."

– "the first few" is a decent idea, since *later* we introduce effects, refinement, and friends

Then the typical thing is:
- **opsem**
- **progress&preservation w.r.t. typing relation**
- **soundness w.r.t. *observational equivalence***

great people prove *completeness/abstraction/full abstraction* in this world

What we do instead is:

- **densem**
- **soundness w.r.t. densem**
- ***completeness* w.r.t. densem**
- **but opsem is subsumed by densem via interaction trees and friends!**

Or alternatively – follow embedded todos?

Much of this thesis consists of variations on a traditional pattern in the PL literature:

- We introduce a language L, along with some informal intuition and motivation
- We give L a formal syntax – usually parametrized by abstract sets of operations and constants
- We then formalize L's "expected" behaviour by equipping it with a simple *operational semantics* **TODO 5: cut below?** – typically, we will "build up" to a full semantics for L, starting with a simple rewriting-based semantics and then adding features like *state* and *nondeterminism* in stages.
- Then, we define *type theory* for L, consisting of:
  - A *type system* – which defines the types a term can have and the contexts in which they are interpreted. Usually, this will be parametrized by an abstract set of *base types*.

    Often, we will study many different languages using the same type system.
  - A *typing relation* – this typically consists of *typing rules* which tell us which terms are well-typed in a given context.

    TODO 6: pull this down? or leave here We usually then want to prove *progress* – that is, that every step in our operational semantics preserves well-typedness.
  - An *equational theory* – this gives us a notion of which terms may be considered *equivalent* within a given context

TODO 7: can pull this down – "we then want to prove soundness, but this is *hard* since observational equivalence is hard – need "equivalent in every context" and that is pain"

we *care* about equivalence because we're writing a *compiler* We usually then want to prove *soundness* – that is, programs which our equational theory identifies as equivalent are "indistinguishable" according to some notion of *observational equivalence*.

TODO 8: denotational semantics – do we pull the operational semantics *down*?

do we even want it at all –
- **it's not part of the MVP...**
- **but it really helps justify our denotational semantics as correct – otherwise completeness seems particularly abstract**

We'll start by giving a formal account of the simplest practical language – that of *first-order* arithmetic expressions, like $x + y$ and $a + (5 \cdot b) + \sin(3)$.

Rather than complicate our language by introducing binary operators (like $x + y$), unary operators (like $-x$), and n-ary function applications (like $f(x, y, z)$), we can instead normalize everything to *S-expressions* – or *sexprs* – of the form $(f\ e_1\ \ldots\ e_n)$. We can then uniformly write:

- binary operations like $x + y$ as binary sexprs $(+\ x\ y)$,
- unary operations $-x$ as unary sexprs $(-\ x)$,
- function appplications $f(x, y, z)$ as n-ary sexprs $(f\ x\ y\ z)$.

In particular, the *operator* f always comes first, and there is no order-of-operations – all bracketing is explicit. Restricting ourselves to *first-order* sexprs means that we always require operators f to be atomic symbols drawn from a fixed set O – in particular, we don't support *partial application*, like $((\mathsf{add}\ 2)\ 3)$, since that would require $f = (\mathsf{add}\ 2)$ to be a compound expression.

Formally, we define our *syntax* as follows:

Definition 3.1 (First-order S-expressions): fixing
- a set of *variables* $x \in N$
- a set of *operations* $f \in O$
- a set of *constants* $c \in V$

we define the set of *first-order S-expressions* $e \in \mathsf{STm}_N(O)$ to be given by the grammar

$$e ::= x \mid c \mid (f\ e^*)$$

We say a sexpr is *closed* if it contains no free variables – otherwise, it is *open* – where the *free variables* $\mathsf{use}(e)$ of a sexpr e are defined as follows:

$$\mathsf{use}(x) = \{x\} \qquad \mathsf{use}(c) = \{\} \qquad \mathsf{use}((f\ e_1\ \ldots\ e_n)) = \bigcup_i \mathsf{use}(e_i)$$

We'll write the set of closed sexprs as $\mathsf{STm}_\emptyset(O)$.

It's easy enough, in the language of simple arithmetic, to define what a closed sexpr "means", since we can just *evaluate* it to find out.

For example, we might have been taught to evaluate expressions from the "inside out", like so:

$$(+\ 5\ (\cdot\ 3\ 7)) = (+\ 5\ 21) = 26$$

We can formalize this intuition in terms of a *small-step operational semantics*: given, for each operation $f \in O$, a (partial) function $\mathsf{ev}(f) : V^* \to V$ telling us what value it returns when called with arguments $c_1, ..., c_n$, we may define a *reduction relation* $e \to e'$ on sexprs as follows:

$$\text{step}\ \frac{\mathsf{ev}(f)(c_1, ..., c_n) = c}{(f\ c_1\ ...\ c_n) \to c} \qquad \text{arg}\ \frac{c_1, ..., c_n \in V \qquad e_1 \to e_1'}{(f\ c_1\ ...\ c_n\ e_1\ ...\ e_m) \to (f\ c_1\ ...\ c_n\ e_1'\ ...\ e_m)}$$

Partiality here means that we don't need to assign a meaning to ill-defined expressions like $(/\ 1\ 0)$ or $(+)$ (when forced, 0 is a decent choice for both of these) – we can simply leave them as undefined.

Notice that we've fixed the *evaluation order* of sexprs to be left-to-right – that is; we have to finish evaluating all terms to the *left* of a given term before we can get to reducing it.

We say a term e *evaluates* to a constant c – written $e \Downarrow c$ – if there exists a finite sequence of reductions

$$e \to e_1 \to ... \to e_n \to c$$

i.e., if $e \to^* c$ – where R^* denotes the *reflexive, transitive closure* of a relation R.

We call a relation $e \Downarrow c$ a *big-step operational semantics* – as our original intuition for evaluation would suggest, c is a pretty good candidate for the "meaning" of a program $e \Downarrow c$.

In particular, we may define a partial function $\mathsf{ev}(e)$ on closed terms by induction as follows:

$$\mathsf{ev}(c) = c \qquad \mathsf{ev}((f\ e_1\ ...\ e_n)) = \mathsf{ev}(f)(\mathsf{ev}(e_1), ..., \mathsf{ev}(e_n))$$

It is straightforward to show that $e \Downarrow \mathsf{ev}(e)$ if and only if $\mathsf{ev}(e)$ is defined.

TODO 9: Consider alternate example: $(\cdot\ (+\ 1\ 1)\ x)$

- **Reduces to** $(\cdot\ 2\ x)$ **– so these should be "the same"**
- **Intuitively the same as** $(+\ x\ x)$
- **Intuitively different from** $(\cdot\ 3\ x)$

We still, however, don't know what an *open* term means: if we try to evaluate

$$(+\ (+\ 3\ 2)\ (\cdot\ x\ 0)) \to (+\ 5\ (\cdot\ x\ 0)) \to\ ???$$

the first time we encounter a variable, the relation defined above becomes *stuck* – that is, we reach a term e which is not a constant, and yet does not reduce to anything – so, according to our current conventions, is ill-defined like $(/\ 1\ 0)$.

If we fire up a Python REPL and try it, it seems to agree:

```
>>> 5 + (x * 0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    5 + (x * 0)
        ^
NameError: name 'x' is not defined
```

On the other hand, if x is in context – say $x = 7$ – this evaluates to 5 as we'd expect:

```
>>> x = 7
>>> 5 + (0 * x)
5
```

What this hints at is that the operational semantics of an open term e depends on the context in which it is evaluated – here, on the values of its free variables.

We can model this by defining a *contextual reduction relation* $\gamma \vdash e \to e'$ parametrized by an *evaulation context* γ – a finitely supported function $\gamma : N \rightharpoonup V$ assigning a values to variables – as follows:

$$\text{var}\ \frac{\gamma(x) = c}{\gamma \vdash x \to c} \qquad \text{step}\ \frac{\mathsf{ev}(f)(c_1, ..., c_n) = c}{\gamma \vdash (f\ c_1\ ...\ c_n) \to c}$$

$$\text{arg}\ \frac{c_1, ..., c_n \in V \qquad \gamma \vdash e_1 \to e_1'}{\gamma \vdash (f\ c_1\ ...\ c_n\ e_1\ ...\ e_m) \to (f\ c_1\ ...\ c_n\ e_1'\ ...\ e_m)}$$

We can then evaluate

$$(x \mapsto 7) \vdash (+\ (+\ 3\ 2)\ (\cdot\ x\ 0)) \to (+\ 5\ (\cdot\ x\ 0)) \to (+\ 5\ (\cdot\ 7\ 0)) \to (+\ 5\ 0) \to 5$$

TODO 10: substitution maps open terms to closed terms

TODO 11: discuss:
- **algebra: "equivalence under substitution"**
- **programming: "observational equivalence"**

Given a set of *types* $A \in \mathcal{T}$, we'll define a *stack-typing relation* on O to be any relation

$$f : [A_1, ..., A_m] \to [B_1, ..., B_n]$$

where

- $A_1, ..., A_m \in \mathcal{T}$ are types – m is called the *input arity* or *arity* of f.
- $B_1, ..., B_n \in \mathcal{T}$ are types – n is called the *output arity* of f.

For now, we'll only consider operations with an output arity of 1.

We define a *(typing) context* $\Gamma \in \mathsf{Ctx}_N(\mathcal{T})$ to be given by a list of *bindings*

$$x_1 : A_1, ..., x_n : A_n$$

We can define a simple type system for S-expressions by giving a typing relation $\Gamma \vdash e : A$ that says "under the assumptions of Γ, the S-expression e has type A". The rules are:

TODO 12: Typing rules for constants – as distinct from nullary ops

$$\text{var-head}\ \frac{}{\Gamma, x : A \vdash x : A} \qquad \text{var-tail}\ \frac{\Gamma \vdash y : B,\ x \neq y}{\Gamma, x : A \vdash y : B}$$

$$\text{app}\ \frac{f : [A_1, ..., A_n] \to [B] \qquad \forall i. \Gamma \vdash a_i : A_i}{\Gamma \vdash (f\ a_1\ ...\ a_n) : B}$$

TODO 13: *Soundness* of typing: preservation of typing

TODO 14: *Completeness* of typing: in the presence of *simple types*

TODO 15: Church vs. Curry – we will only consider *well-typed* terms

TODO 16: Doesn't lose generality – unityping works in this framework due to multi-arity

TODO 17: *Substitution* and *observational equivalence* – typing rules for substitution – two open programs are equivalent iff equivalent under every substitution – this is usually where *metavariables* come in

TODO 18: *Purity* – required for substitution to preserve equivalence – typing rules for purity

TODO 19: Denotational semantics of S-expressions: partial functions

TODO 20: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 21: Equational theory of S-expressions: *purity* – as otherwise you can't *introduce* divisions

TODO 22: Equational theory of S-expressions: *soundness* and *completeness*

4. State

TODO 23: Operational semantics of S-expressions – *state*

TODO 24: *Observational equivalence* in the presence of state

TODO 25: Denotational semantics of S-expressions – the partial state monad

TODO 26: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 27: Equational theory of S-expressions: *purity*

TODO 28: Equational theory of S-expressions: *soundness* and *completeness*

5. Sequencing

6. Branching

7. Loops

8. Unstructured Control-Flow

Bibliography

1. Sample Appendix

SAMPLE TEXT